Name:

PRN:

Experiment no 10

Title: CUDA Optimization and Parallel Algorithms: Shared Memory, Reduction, Scan and Histogram

1. Aim:

To understand and implement two complementary aspects of GPU programming using CUDA:

1. **CUDA Memory Optimization** — optimizing memory access using shared memory and tiling.
2. **CUDA Parallel Algorithms** — designing and implementing fundamental parallel algorithms such as reduction, prefix sum (scan), and histogram.

Students will compare different CUDA implementations with sequential versions and analyze the impact of memory access, synchronization, thread organization, and atomic operations on performance.

2. Why Are We Learning This?

Writing a CUDA program that runs on a GPU is only the first step. To obtain good performance, we need to understand how data moves through the GPU memory hierarchy and how algorithms can be decomposed into independent parallel operations.

For example:

- Matrix multiplication may repeatedly access the same matrix elements.
- Reduction requires many threads to combine their partial results.
- Prefix sum contains data dependencies that make straightforward parallelization challenging.
- Histogram computation may involve many threads updating the same memory locations.

These problems expose important GPU concepts such as:

- Global memory
- Shared memory
- Memory locality
- Tiling

- Synchronization
- Atomic operations
- Thread cooperation
- Parallel decomposition
- Memory contention

These concepts are fundamental to GPU applications in AI, deep learning, computer vision, scientific computing, simulations, and data analytics.

3. CUDA in HPC and AI:

CUDA-based optimization and parallel algorithms are widely used in:

Artificial Intelligence

- Matrix multiplication in neural networks
- Tensor operations
- Feature extraction
- Batch processing
- Deep-learning training and inference

Computer Vision

- Image filtering
- Histogram computation
- Convolution
- Image enhancement
- Object detection

Scientific Computing

- Numerical simulations
- Linear algebra
- Molecular dynamics
- Computational fluid dynamics

Data Analytics

- Sorting
- Reduction
- Prefix-sum operations
- Histogram generation
- Large-scale statistical processing

Key Insight

High GPU performance depends not only on the number of threads but also on how efficiently those threads access and share data.

4. Learning Focus

By completing this assignment, you will learn:

- Global-memory-based CUDA computation
- Shared-memory programming
- Tiled matrix multiplication
- CUDA synchronization using `__syncthreads()`
- Parallel reduction
- Prefix sum / scan
- Histogram computation
- Atomic operations
- Memory-access optimization
- Kernel execution time measurement
- Kernel optimization and performance comparison

5. Pre-Lab Activity

Before starting the programming tasks, answer the following.

Q1. Differentiate between:

- Host
- Device
- Global memory
- Shared memory
- Registers

Q2. Why is shared memory generally faster than global memory?

Q3. What is meant by **memory coalescing**?

Q4. Why is `__syncthreads()` required when threads cooperate through shared memory?

Q5. What is a race condition?

Q6. Why are atomic operations required in some histogram implementations?

Q7. What is the difference between:

- Reduction
- Prefix sum
- Histogram

Q8. Why is prefix sum more challenging to parallelize than vector addition?

6. Software and Hardware Requirements

Software

- NVIDIA CUDA Toolkit
- `nvcc`
- C/C++ compiler
- Linux/Windows environment
- Nsight Systems or Nsight Compute, if available

Hardware

- NVIDIA CUDA-capable GPU
- Recommended: GPU with sufficient global and shared memory

Record the following information:

Parameter	Your System
CPU	
CPU cores	
RAM	
GPU model	
GPU memory	
Compute Capability	
CUDA version	
Operating System	
Compiler version	

PART I — CUDA MEMORY OPTIMIZATION

7. Task 1 – Matrix Multiplication Using Global Memory

Problem

Implement matrix multiplication:

Implement

Version 1 – Sequential CPU

Implement matrix multiplication using standard C/C++.

Version 2 – CUDA Global Memory

Implement matrix multiplication using CUDA where each thread computes one element of matrix `C`.

Use:

```
row = blockIdx.y * blockDim.y + threadIdx.y
col = blockIdx.x * blockDim.x + threadIdx.x
```

Experiment

Use different matrix sizes such as:

- 256 × 256
- 512 × 512
- 1024 × 1024
- Larger size if resources permit

Use block configurations such as:

- 8 × 8
- 16 × 16
- 32 × 32

Record

- CPU execution time
- CUDA kernel time

- CUDA total execution time
- Speedup
- GFLOPS, if possible

8. Task 2 – Tiled Matrix Multiplication Using Shared Memory

Modify the matrix multiplication program so that frequently reused matrix elements are loaded into shared memory.

For each block:

1. Load a tile of `A` into shared memory.
2. Load a tile of `B` into shared memory.
3. Synchronize threads.
4. Perform partial multiplication.
5. Synchronize again.
6. Move to the next tile.

Conceptually:

```
Global Memory
     ↓
Shared Memory Tile
     ↓
Parallel Computation
     ↓
Shared Memory Tile
     ↓
Next Tile
```

Use:

```
__shared__ float tileA[TILE][TILE];
__shared__ float tileB[TILE][TILE];
```

and appropriate synchronization.

Experiment with

- TILE = 8
- TILE = 16
- TILE = 32

Compare the performance of different tile sizes.

9. Task 3 – Global Memory vs Shared Memory

Compare the two implementations:

Version A

CUDA matrix multiplication using global memory.

Version B

CUDA tiled matrix multiplication using shared memory.

Prepare a table:

Matrix Size	Block/Tile	Global Memory Time	Shared Memory Time	Speedup
256 × 256	8/16/32			
512 × 512	8/16/32			
1024 × 1024	8/16/32			

Analyze

1. Why does shared memory improve performance?
2. Does shared memory always provide the same improvement?
3. What happens when the tile size changes?
4. What limitations does shared memory have?
5. How does data reuse affect performance?

PART II — CUDA PARALLEL ALGORITHMS

10. Task 4 – Parallel Reduction

Reduction combines many input values into a smaller number of results.

For example:

Sum of first 100000 numbers

Implement:

Version A

Sequential sum on CPU.

Version B

CUDA reduction using global memory.

Version C

CUDA reduction using shared memory.

A typical reduction pattern is:

```
x0 x1 x2 x3 x4 x5 x6 x7

 ↓  ↓  ↓  ↓

x0+x1   x2+x3   x4+x5   x6+x7

    ↓       ↓       ↓

 (x0+x1)+(x2+x3)
       ...
```

Use multiple blocks for large arrays and combine block-level results.

Extend the program

Modify the reduction program to calculate:

- Sum

- Maximum
- Minimum

Experiment

Use:

- (N = 10^5)
- (N = 10^6)
- (N = 10^7)
- (N = 10^8), if resources permit

Compare:

- CPU
- CUDA global-memory reduction
- CUDA shared-memory reduction

11. Task 5 – Parallel Prefix Sum / Scan

A prefix sum transforms:

```
Input:
3  1  7  0  4  2
```

into an inclusive scan:

```
3  4  11  11  15  17
```

Implement a CUDA-based prefix sum.

Step 1 – Sequential Version

Implement prefix sum on CPU.

Step 2 – CUDA Version

Implement a parallel scan using shared memory.

you may implement a simple tree-based scan such as the **Hillis–Steele** approach.

For example:

```
Initial:
a0 a1 a2 a3 a4 a5 a6 a7

Step 1:
a0
a0+a1
a1+a2
a2+a3
...

Step 2:
a0
a0+a1
a0+a1+a2
a1+a2+a3
...
```

Use appropriate synchronization between stages.

Analyze

- Number of synchronization stages
- Number of operations
- Memory accesses
- Execution time
- Scalability

Experiment

Test different array sizes and compare CPU and CUDA execution.

12. Task 6 – CUDA Histogram

A histogram counts the frequency of values.

For example:

```
Input:
1 2 1 3 2 1 4 3 2
```

Histogram:

```
Value     Frequency
0            0
1            3
2            3
3            2
4            1
```

Implement a CUDA histogram.

Version A

Sequential CPU histogram.

Version B

CUDA histogram using global-memory atomic operations.

Use:

```
atomicAdd(&histogram[value], 1);
```

Version C – Advanced

Implement histogram using **shared memory** within each block and combine the block-level histograms into the final result.

Compare:

- Global-memory atomic histogram
- Shared-memory histogram

Analyze

1. Why can atomic operations become a bottleneck?
2. What happens when many threads update the same bin?

3. How does shared memory reduce contention?
4. What happens when the number of histogram bins increases?

13. Task 7 – Comparative Performance Analysis

Compare the major CUDA implementations developed in this assignment.

Algorithm	CPU	CUDA Basic	CUDA Optimized	Main Optimization
Matrix Multiplication				Shared memory
Reduction				Shared memory
Prefix Sum				Parallel scan
Histogram				Shared memory/atomics

Calculate:

Speedup

Efficiency

For CUDA, also distinguish between:

- Kernel execution time
- Host-to-device transfer time
- Device-to-host transfer time
- Total GPU application time

14. Performance Investigation

For each algorithm, investigate the effect of at least two parameters.

Matrix Multiplication

- Matrix size
- Tile size

Reduction

- Array size
- Threads per block

Prefix Sum

- Array size
- Block size / scan configuration

Histogram

- Input size
- Number of bins

Prepare suitable graphs.

Recommended graphs

1. Matrix size vs execution time
2. Tile size vs matrix multiplication time
3. Input size vs reduction time
4. Input size vs scan time
5. Input size vs histogram time
6. CPU vs CUDA speedup

15. Important CUDA Concepts to Identify

While completing the experiments, identify where each concept is used.

CUDA Concept	Where Used
Threads	All kernels

Blocks	All kernels
Grid	All kernels
Global memory	Matrix multiplication/reduction
Shared memory	Tiled multiplication/reduction/scan
Synchronization	Reduction/scan/tiling
Atomic operations	Histogram
Memory locality	Matrix multiplication
Data reuse	Tiled multiplication
Race condition	Histogram/reduction
Parallel decomposition	All algorithms

16. Section C – Challenge Tasks

attempt **at least one** of the following.

Challenge 1 – Optimized Reduction

Improve the reduction implementation by reducing unnecessary global-memory accesses and synchronization.

Compare:

```
Naive reduction
       ↓
Shared-memory reduction
       ↓
Optimized reduction
```

Explain every optimization introduced.

Challenge 2 – Multi-Block Prefix Sum

Extend the prefix-sum implementation to arrays larger than a single CUDA block.

Design a solution using:

```
Input Array
     ↓
Block-wise Scan
     ↓
Block Sums
```

```
     ↓
Scan of Block Sums
     ↓
Add Offsets
     ↓
Final Prefix Sum
```

Challenge 3 – Optimized Histogram

Implement:

```
Global atomic histogram
        vs
Shared-memory block histogram
```

Study how performance changes when:

- input size increases
- number of bins changes
- data distribution changes

Test at least two distributions, such as:

- Uniform distribution
- Highly concentrated distribution

Explain how data distribution affects atomic contention.

Challenge 4 – CUDA Application: 2D Convolution

Implement a simple 2D convolution using CUDA.

Compare:

1. CPU implementation
2. CUDA global-memory implementation
3. CUDA shared-memory implementation

This serves as an application of the concepts learned in this assignment.

17. AI/HPC Case Study

GPU Acceleration in Deep Learning

Modern neural networks perform a very large number of mathematical operations.

A simplified neural-network layer can be represented as:

[
Y = WX + b
]

where matrix multiplication forms a major computational component.

Discuss how the concepts learned in this assignment contribute to GPU acceleration:

Matrix Multiplication

Used in:

- Fully connected layers
- Attention mechanisms
- Transformer models
- Embedding operations

Reduction

Used in:

- Loss calculation
- Statistical operations
- Normalization
- Aggregation

Prefix Sum

Used in:

- Data compaction
- Sparse operations
- GPU scheduling algorithms
- Graph processing

Histogram

Used in:

- Image processing
- Feature analysis
- Quantization
- Data preprocessing

Why do modern AI frameworks such as PyTorch and TensorFlow rely heavily on optimized GPU kernels and libraries rather than simply executing every operation as a naïve CUDA kernel?

18. Questions

Conceptual

1. Why is shared memory faster than global memory?
2. Why does matrix multiplication benefit significantly from tiling?
3. Why is synchronization necessary when using shared memory?
4. What is a race condition?
5. Why does reduction require synchronization?
6. Why is prefix sum more difficult than vector addition?
7. Why can atomic operations reduce histogram performance?
8. Why does data distribution affect histogram performance?

Analytical

9. Does increasing the number of CUDA threads always improve performance? Explain using your observations.
10. Does increasing the tile size always improve matrix multiplication performance?
11. Why might a CUDA implementation perform worse than a CPU implementation for a small input?
12. Why should kernel execution time and data-transfer time be measured separately?
13. Which of the implemented algorithms is most sensitive to synchronization? Explain based on your implementation.
14. Which optimization produced the largest performance improvement in your experiments? Support your answer using measured data.

19. Reflection

After completing the assignment, answer:

1. What was the most important difference between global and shared memory that you observed?
2. Which algorithm was easiest to parallelize? Why?
3. Which algorithm was most difficult to parallelize? Why?
4. What type of workload benefits most from shared memory?
5. What type of workload may suffer from atomic contention?
6. What optimization would you investigate next if you had access to a more powerful GPU?

20. Deliverables

Submit:

1. Source Code

- CPU matrix multiplication
- CUDA global-memory matrix multiplication
- CUDA shared-memory matrix multiplication
- CPU reduction
- CUDA reduction
- CPU prefix sum
- CUDA prefix sum
- CPU histogram
- CUDA histogram
- Challenge implementation, if attempted

2. Experimental Data

Provide tables containing:

- Input size
- Block size
- Tile size
- Execution time
- Kernel time
- Transfer time
- Speedup
- Efficiency, where applicable

3. Graphs

Include relevant performance graphs.

4. Analysis

Explain:

- Global vs shared memory
- Tiling
- Synchronization
- Reduction strategy
- Scan strategy
- Atomic operations
- Performance bottlenecks

5. Screenshots

Include:

- GPU information
- Compilation
- Program execution
- Performance measurements

6. Short Technical Conclusion

Summarize the major performance observations from all experiments.